

SIMATS ENGINEERING

CO5-ASSESSMENT TOOL3-Reverse Engineering

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Register Number	192311097
Name	Dillilokesh D

COURSE OUTCOME COVERED

CO5: Design intelligent agents and real-time AI-based systems (chatbots, expert systems, emotion detection, edge AI, autonomous drones, and related applications) by selecting appropriate agent architectures, knowledge representation, and reasoning mechanisms to solve real-world problems. (BL6)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:50

Q1. The following is a conversation between a user and an AI-based flight-booking chatbot. Study the transcript given below and reverse-engineer the chatbot's design by: (a) writing its PEAS description, (b) classifying the type of agent it represents with justification, (c) identifying the likely component (NLU, Dialogue Manager, Knowledge Base, or NLG) responsible for each chatbot response, and (d) suggesting one enhancement using a learning-agent architecture.

Speaker	Message
User	I want to book a flight to Chennai next Friday.
Chatbot	Sure! What time would you like to depart, and do you have a preferred airline?
User	Morning flight, no preference.
Chatbot	I found 3 morning flights to Chennai on Friday. Shall I show you the cheapest option first?

Q2. The block diagram below shows the processing pipeline of a Real-Time Emotion Detection System. Reverse-engineer the system by: (a) explaining the function performed at each stage of the pipeline, (b) identifying the likely AI/ML technique used at each stage, (c) writing the PEAS description of the system, and (d) classifying the

Real-Time Emotion Detection System — processing pipeline



agent type (e.g., simple reflex, model-based) with justification.

Q3. An Expert System Shell for medical diagnosis contains the rule base, input facts, and reasoning trace given below. Study it and reverse-engineer the shell by: (a) identifying the type of chaining (forward/backward) used to reach the diagnosis, with justification, (b) identifying the expert-system-shell components illustrated (knowledge base, inference engine, explanation facility), (c) explaining how the shell would perform knowledge acquisition to add a new rule for dengue, and (d) explaining, with reasoning, whether backward chaining could reach the same diagnosis.

Rule	IF (condition)	THEN (conclusion)
R1	fever = yes AND cough = yes	suspect = flu
R2	suspect = flu AND body_ache = yes	diagnosis = Influenza
R3	fever = yes AND rash = yes	suspect = measles

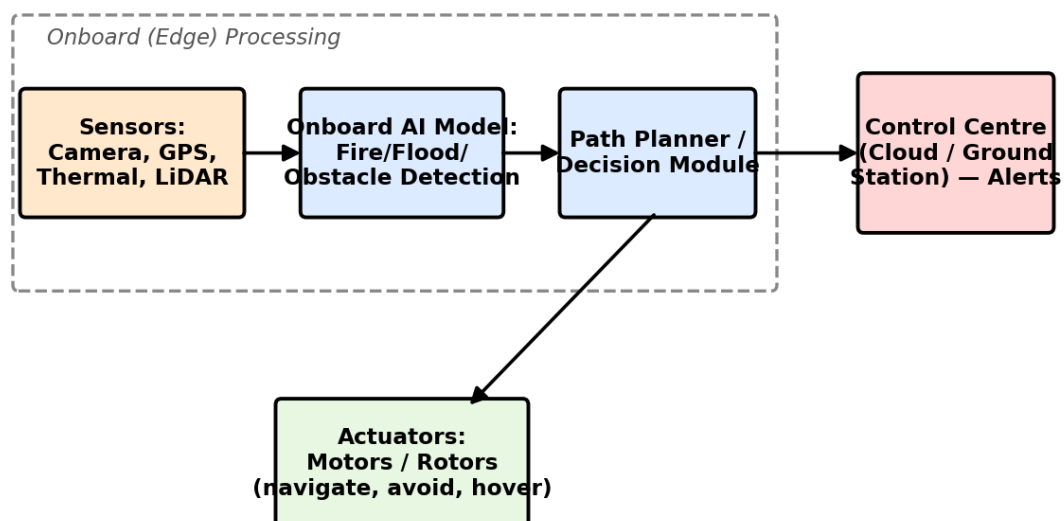
Given facts: fever = yes, cough = yes, body_ache = yes

Reasoning trace: R1 fires first (fever & cough matched) → suspect = flu. R2 then fires (suspect = flu & body_ache matched) → diagnosis = Influenza.

Explanation generated by the shell: “Diagnosis = Influenza because fever and cough indicated flu, and flu together with body ache confirmed Influenza.”

Q4. The diagram below shows the system architecture of an Autonomous Drone used for Disaster Detection with Edge AI deployment. Reverse-engineer the design by: (a) writing the PEAS description of the drone agent, (b) classifying the agent type (goal-based/utility-based) with justification, (c) identifying which components run on the edge versus the cloud and justifying why on-edge processing was chosen for detection and obstacle avoidance, and (d) identifying whether the agent's control architecture is reactive, deliberative, or hybrid, with justification.

Autonomous Drone — Disaster Detection (Edge AI) system architecture



Q5. The table below shows sample output logs from an AI-based Deepfake & Face Authenticity Verification system across five video frames. Study the output and reverse-engineer the system by: (a) identifying the likely input features used by the model (e.g., blink rate, texture/artifact score), (b) identifying the type of model/technique likely used to produce these outputs, (c) explaining how the final Real/Fake decision may have been derived from the frame-level confidence scores,

and (d) suggesting one way a swarm/ensemble-based approach could improve the system's reliability.

Frame No.	Blink Rate (per 10s)	Texture/Artifact Score	Fake Confidence (%)	Prediction
1	4	0.12	8	Real
2	1	0.61	74	Fake
3	0	0.83	91	Fake
4	5	0.09	5	Real
5	2	0.55	68	Fake

Overall system verdict: Fake (3 of 5 frames flagged — majority vote)

Code of Conduct:

I Dillilokesh D 192311097 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Dillilokesh D

RUBRICS FOR CALCULATION

Criteria	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning	Max Marks
Identification of Agent Type & PEAS (correctly inferred from the given artifact)	Correctly identifies the agent type and gives a complete, accurate PEAS description, fully consistent with the given artifact.	Identifies the agent type and PEAS description correctly with only minor omissions.	Partially correct identification of agent type/PEAS; some elements are missing or inconsistent with the artifact.	Agent type/PEAS identification is largely incorrect or not attempted.	10
Identification of Architecture & Components (mapping artifact evidence to system components)	Accurately identifies all relevant architectural components and correctly maps each to specific evidence in the artifact.	Identifies most components correctly with reasonable mapping to the artifact; minor gaps present.	Identifies some components; mapping to the artifact is weak or partially incorrect.	Little or no correct identification of architecture/components.	10
Identification of Underlying Technique/Algorithm/Reas	Correctly reverse-engineers the technique, algorithm, or	Correctly identifies the technique/mechanism	Identification is only partially correct or lacks	Technique/mechanism is incorrectly identified or not addressed.	10

Criteria	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning	Max Marks
oning Mechanism	reasoning mechanism (e.g., chaining type, ML technique) with clear supporting evidence.	with adequate, mostly accurate justification.	sufficient supporting evidence.		
Quality of Justification & Use of Evidence (linking conclusions back to the given artifact)	Every conclusion is clearly justified with specific, accurate references to the given artifact.	Most conclusions are justified with reasonable references to the artifact.	Justification is vague or only loosely connected to the artifact.	Conclusions are stated with little or no justification or evidence.	10
Clarity & Completeness of Presentation (addresses all sub-parts of the question in an organised manner)	All sub-parts are addressed completely, in a clear, well-organised, and logically structured manner.	Most sub-parts are addressed clearly with minor gaps in organisation or completeness.	Some sub-parts are missing or the presentation is unclear/poorly organised.	Response is incomplete, disorganised, or largely missing.	10
				Total	50

ANSWERS:

Question 1 — AI-Based Flight-Booking Chatbot

The transcript shows a two-turn exchange in which the user asks to book a flight to Chennai for next Friday, the chatbot asks for the missing time and airline preference, the user answers with "morning, no preference", and the chatbot reports back that three morning flights were found and offers to show the cheapest one. The design below is inferred entirely from this behaviour.

(a) PEAS Description

Element	Description
Performance measure	Successful completion of a valid booking; minimum number of conversational turns needed to collect all required slots (destination, date, time, airline); relevance/accuracy of the flights returned; price-consciousness of the default suggestion ("cheapest option first"); response latency; user satisfaction and task-completion rate.
Environment	A natural-language, turn-taking conversational environment layered on top of a live flight-inventory system. It is partially observable (the chatbot cannot see the user's full intent, budget, or urgency, only what is typed), dynamic (seat availability and fares change independently of the conversation), and single-agent from the chatbot's perspective (one user per session), though the underlying airline system may itself be multi-agent.
Actuators	The natural-language response generator (the printed chatbot messages); presentation of flight options/cards; the ability to issue a query against the airline's flight-search/booking API.
Sensors	The text/speech input channel through which the user's utterances arrive, and the parsed intents/entities (destination, date, time-of-day, airline preference) extracted from that text.

(b) Agent Type Classification

This chatbot is best classified as a goal-based agent implemented through slot-filling dialogue management, with a thin model-based layer. It is not a simple reflex agent, because its second response ("I found 3 morning flights...") does not follow directly and only from the immediately preceding user utterance — it follows from the accumulated state of the booking frame (destination = Chennai, date = Friday, time = morning, airline = none), which the chatbot must have been maintaining since turn one. That accumulated, non-percept-visible state is exactly the internal model that distinguishes a model-based agent from a simple reflex agent.

It is also goal-based rather than purely model-based, because every question it asks is explicitly chosen to close the gap between the current dialogue state and the goal state "all slots filled and a flight confirmed" — asking for time and airline is a sub-goal generated because a booking cannot be completed without them. There is a faint trace of utility-based behaviour in the offer to show the "cheapest option first" (a default ranking heuristic over multiple valid results), but with only one data point this looks like

a fixed business rule rather than a learned or optimised utility function, so utility-based is not the primary classification.

(c) Component Responsible for Each Response

A typical task-oriented dialogue system is built from four cooperating components: an NLU module that turns raw text into structured intents/entities, a Dialogue Manager (DM) that tracks state and decides the next action, a Knowledge Base (KB) that holds or retrieves domain data such as flight inventories, and an NLG module that turns the DM's chosen action into natural language. Mapping the transcript onto this pipeline:

Chatbot Turn	NLU	Dialogue Manager	Knowledge Base	NLG
"Sure! What time would you like to depart, and do you have a preferred airline?"	Extracts intent = BookFlight and entities {destination: Chennai, date: next Friday} from the user's first message.	Checks the booking frame, finds time and airline slots empty, and selects the dialogue act request_slot(time, airline).	Not yet queried — no search is possible until the mandatory slots are filled.	Renders the chosen dialogue act into the fluent question shown, combining two slot requests into one sentence.
"I found 3 morning flights to Chennai on Friday. Shall I show you the cheapest option first?"	Extracts time = morning and airline = none/any from "Morning flight, no preference."	Confirms all mandatory slots are now filled, decides to trigger a KB lookup, then chooses the dialogue act present_results(count) + confirm(show_cheapest).	Executed here: the frame (Chennai, Friday, morning) is sent to the flight-search/inventory system, which returns a result count (3) and the cheapest fare's details.	Composes the count and destination/date into the results sentence and phrases the next-step offer as a yes/no question.

(d) Enhancement Using a Learning-Agent Architecture

A classical learning agent adds four elements around the existing performance element: a Critic that scores outcomes against a performance standard, a Learning Element that updates the agent's behaviour from that feedback, and a Problem Generator that occasionally proposes exploratory actions. Applied here, the Critic would observe implicit and explicit signals — did the user accept the cheapest-first suggestion, how many turns the booking took, and whether the session ended in a completed booking or an abandoned one. The Learning Element would use this feedback to train a dialogue policy (e.g., via reinforcement learning, where the reward is +1 for a completed booking and a small negative reward per extra turn) that decides, for a given user, whether to ask for time and airline together (as shown) or to skip straight to showing options when a returning user's past bookings suggest they never care about airline. The Problem Generator would occasionally try a different elicitation order or a different default ranking (e.g., fastest-first instead of cheapest-first) on a small fraction of sessions purely to gather data on whether it improves completion rate, turning the static slot-filling script into one that keeps improving from real usage rather than remaining a fixed hand-written policy.

Question 2 — Real-Time Emotion Detection System

(a) Function of Each Pipeline Stage & (b) Likely AI/ML Technique

Stage	Function Performed	Likely AI/ML Technique
1. Video/Image Acquisition	Continuously captures live frames from a camera sensor and hands them to the pipeline at a fixed frame rate.	Not a learned technique — raw sensor I/O and frame buffering/sampling.
2. Face Detection & Localisation	Locates and crops the bounding region(s) of any face present in the frame, discarding background.	Classical Haar-cascade/Viola–Jones detectors, or modern deep detectors such as MTCNN/SSD-based face detectors.
3. Pre-processing	Aligns the face (using eye/landmark positions), normalises scale, lighting and colour, and resizes to the input size the classifier expects.	Geometric image transforms plus a facial-landmark regressor (e.g., an ensemble of regression trees, as in Dlib) for alignment points.
4. Feature Extraction	Converts the normalised face image into a compact numerical representation that captures the visual cues relevant to expression (eyebrows, mouth curvature, eye openness, wrinkling).	A convolutional neural network (CNN) used as a feature embedder (e.g., a ResNet/MobileNet backbone), or classical descriptors such as HOG/LBP in lighter-weight systems.
5. Emotion Classification	Maps the extracted features onto one of a fixed set of emotion categories (happy, sad, angry, surprised, neutral, etc.) with a confidence score.	A softmax classification head on the CNN, or a separate classifier (SVM/Random Forest) trained on the extracted features; an LSTM/temporal model if a short frame sequence rather than a single frame is classified.
6. Post-processing & Output	Smooths the label across a short window of frames to remove jitter, applies a confidence threshold, and renders/forwards the final emotion label.	Temporal smoothing (moving average / majority vote over a sliding window) — a light statistical technique rather than a learned model.

(c) PEAS Description

Element	Description
Performance measure	Classification accuracy against ground-truth emotion labels, real-time throughput (frames processed per second), robustness to pose/lighting/occlusion variation, and low false-positive/false-negative rate on the emotion label.
Environment	A live, real-world video feed of one or more human faces. It is partially observable (the true internal emotional state is never directly observable, only its facial expression), dynamic (lighting, head pose and motion change continuously), continuous in time, and potentially multi-agent when several faces appear in one frame.
Actuators	The rendered emotion label/confidence overlay on the video feed, and any downstream trigger it drives (e.g., logging, an adaptive-UI response, or an alert to a human operator).
Sensors	The camera (RGB, and sometimes an infrared/depth sensor for low-light robustness) supplying the raw frame stream.

(d) Agent Type Classification

This system is best classified as a model-based reflex agent. It cannot be a pure simple reflex agent because the post-processing stage smooths the output over a window of previous frames — the current output depends on an internal state built from percept history, not solely on the current percept, which is precisely the feature that separates a model-based agent from a simple reflex agent. At the same time, it has no explicit long-horizon goal it is trying to achieve, no planning over future actions, and no utility function trading off competing outcomes — each processing cycle simply maps the (state-augmented) percept to an output label. That combination — internal state derived from percept history, but no goal/utility reasoning — places it in the model-based reflex category rather than goal-based or utility-based.

Question 3 — Expert System Shell for Medical Diagnosis

(a) Type of Chaining Used

The reasoning trace uses forward chaining (data-driven reasoning). The shell begins with the known facts — fever = yes, cough = yes, body_ache = yes — and scans the rule base for any rule whose IF-conditions are already satisfied. R1's condition (fever = yes AND cough = yes) matches immediately, so it fires and asserts the new fact suspect = flu into working memory. That new fact then makes R2's condition (suspect = flu AND body_ache = yes) satisfiable, so R2 fires and asserts diagnosis = Influenza. This progression — facts trigger rules, rules add facts, new facts trigger further rules — is the defining recognise-act cycle of forward chaining. A backward-chaining trace would instead have started from a hypothesis such as "is diagnosis = Influenza true?" and worked backwards to see which facts would need to hold; nothing in the given trace starts from a goal, so forward chaining is the only reading consistent with the evidence.

(b) Expert-System-Shell Components Illustrated

Component	How It Is Illustrated
Knowledge Base	The three production rules R1–R3, encoding the physician's domain expertise as IF–THEN statements independent of any particular patient's data.
Working Memory / Fact Base	The given facts (fever = yes, cough = yes, body_ache = yes) together with the facts derived during reasoning (suspect = flu, diagnosis = Influenza).
Inference Engine	The match–select–execute cycle visible in the trace: matching facts against rule conditions, selecting R1 then R2 to fire (conflict resolution), and executing each rule's THEN clause to update working memory.
Explanation Facility	The generated sentence — “Diagnosis = Influenza because fever and cough indicated flu, and flu together with body ache confirmed Influenza” — which is built by walking back through the chain of fired rules (R2 then R1) and translating each rule's condition/conclusion pair into natural language, giving the classic “why/how” justification expected of an expert-system shell.

(c) Knowledge Acquisition for a New Dengue Rule

Because a shell keeps the inference engine completely separate from the domain-specific rules, adding dengue support means extending only the knowledge base — the reasoning engine itself needs no code changes. The acquisition process would typically proceed as follows:

- Elicitation: a domain expert (physician) is interviewed, or existing case records/decision tables are studied, to identify the attributes that distinguish dengue from the conditions already modelled — for example joint_pain, rash, and a lab attribute such as platelet_count.
- Formalisation: the elicited criteria are written in the shell's existing rule syntax, e.g. R4: IF fever = yes AND joint_pain = yes AND rash = yes THEN suspect = dengue, and R5: IF suspect = dengue AND platelet_count = low THEN diagnosis = Dengue — mirroring the two-stage suspect-then-diagnosis pattern already used for flu.
- Conflict checking: because R3 already fires suspect = measles whenever fever = yes AND rash = yes, the knowledge engineer must check whether the new dengue rule can co-fire with R3 on the same facts and, if so, add a distinguishing attribute (joint_pain) or a conflict-resolution strategy (e.g., rule specificity or certainty factors) so the two suspicions do not silently collide.
- Encoding: the new rule(s) are entered into the knowledge base through the shell's rule editor/knowledge-acquisition interface, without touching the inference engine or explanation facility, which will automatically pick up and explain the new rule the next time it fires.
- Validation: the updated rule base is run against known historical cases (including the existing flu and measles cases) to confirm the new rule fires correctly on true dengue cases and does not introduce false positives on the older cases — a standard verification/refinement loop before the rule is trusted in production.

(d) Could Backward Chaining Reach the Same Diagnosis?

Yes. Backward chaining would start from the hypothesis “diagnosis = Influenza” and work backwards through the rules that could prove it. To prove diagnosis = Influenza it needs R2's condition, suspect = flu AND body_ache = yes; body_ache = yes is already a known fact, so the only remaining sub-goal is to prove suspect = flu. To prove that, it needs R1's condition, fever = yes AND cough = yes, both of which are already known facts. Since every sub-goal along the chain resolves to a known fact with no missing information, backward chaining succeeds and reaches exactly the same diagnosis = Influenza.

The two strategies differ in efficiency and side effects rather than in the final answer for this case: forward chaining evaluates every rule whose conditions are satisfied, so if rash had also been yes it would additionally derive suspect = measles from R3 as an (irrelevant) side conclusion; backward chaining, being driven purely by the single hypothesis “Is it Influenza?”, would never even inspect R3, since R3 plays no part in proving that specific goal. This makes backward chaining preferable when a clinician already has one suspected diagnosis to confirm, and forward chaining preferable when the system must discover, from raw symptoms alone, everything the data could imply.

Question 4 — Autonomous Drone for Disaster Detection (Edge AI)

(a) PEAS Description

Element	Description
Performance measure	Speed and accuracy of disaster-indicator detection (fire, smoke, flooding, stranded survivors); zero collisions during navigation; area of the disaster zone covered per flight; battery/energy efficiency; timeliness of alerts relayed to the human command centre; and a low false-alarm rate.

Element	Description
Environment	An outdoor disaster zone: partially observable (sensors see only a local field of view and cannot perceive the whole terrain at once), dynamic (fire spread, moving debris, other aircraft), unstructured and often GPS-degraded, and non-deterministic due to weather and damaged infrastructure.
Actuators	Rotor/motor controls (throttle, pitch, roll, yaw) for flight; the camera/gimbal orientation; the communication transmitter used to relay alerts, imagery and telemetry to the ground station; optionally a payload-release mechanism for supply drops.
Sensors	RGB and thermal cameras, LIDAR/ultrasonic proximity sensors, GPS, an inertial measurement unit (accelerometer/gyroscope), and a barometer for altitude.

(b) Agent Type Classification

The drone is best classified as a utility-based agent built on top of goal-based planning. It is goal-directed in the obvious sense — reach the search area, scan it, return to base before the battery is exhausted — which already requires planning/path-finding rather than reflexive action. It is specifically utility-based, and not merely goal-based, because the goals compete: maximising area coverage, minimising battery consumption, minimising collision risk, and prioritising the most likely disaster hotspots cannot all be maximised simultaneously, so the drone must weigh them against one another (e.g., is it worth a longer detour to fully image a promising thermal hotspot at the cost of remaining flight time?). Choosing the best action under such trade-offs, rather than any action that merely satisfies the goal, is the defining behaviour of a utility-based agent.

(c) Edge vs Cloud Components, and Why Detection/Avoidance Run on the Edge

Tier	Components Running There
Edge (onboard the drone)	Real-time disaster-indicator detection (a lightweight CNN such as MobileNet/Tiny-YOLO classifying fire, smoke, flood or survivors from the camera feed); obstacle avoidance (depth estimation/SLAM feeding a reactive collision-avoidance controller); low-level flight stabilisation (PID control loops); emergency manoeuvring.
Cloud / Ground Station	Aggregation of data from one or many drones; the human responder's dashboard and visualisation; long-term storage of imagery/telemetry; retraining and distribution of updated detection models; fleet-level mission planning; non-time-critical damage-assessment analytics.

On-edge processing is the only viable choice for detection and obstacle avoidance for three reasons. First, latency: avoiding a collision or reacting to a sudden hazard requires a response within milliseconds, and a round trip to a cloud server (even ignoring network congestion) cannot reliably meet that deadline. Second, connectivity: disaster zones routinely have damaged or overloaded communication infrastructure, so any safety-critical function that depended on a live network link could fail exactly when it is needed most — onboard inference lets the drone keep flying safely even fully disconnected. Third, bandwidth and cost: continuously streaming raw high-resolution video to the cloud for every navigation decision is far more bandwidth-hungry than transmitting occasional compressed detections and summary telemetry, which is all the cloud tier actually needs.

(d) Control Architecture Classification

The drone's control architecture is hybrid, combining a reactive layer with a deliberative layer. Obstacle avoidance is reactive: it maps sensor input almost directly to a corrective actuator command with no

planning involved, functioning like the lowest layer of a subsumption architecture that exists purely to guarantee safety. Mission-level behaviour — planning a coverage path over the search area, sequencing which hotspots to visit first, and deciding when to abort and return to base — is deliberative, since it requires a world model, an explicit plan spanning many future actions, and reasoning about trade-offs. Layering a fast, safety-critical reactive controller underneath a slower, model-based deliberative planner is the standard hybrid design used in real autonomous UAV systems, and is consistent with everything inferred in parts (a)–(c) above.

Question 5 — Deepfake & Face Authenticity Verification System

The five logged frames each report a blink rate, a texture/artifact score, a fake-confidence percentage, and a binary prediction. Reading the numbers together: low blink rate consistently pairs with high texture score and high fake confidence (frames 2, 3, 5), while high blink rate pairs with low texture score and low fake confidence (frames 1, 4) — and every confidence value sits cleanly on one side of the 50% mark of its corresponding label.

Reference Log (as given)

Frame	Blink Rate (/10s)	Texture/Artifact Score	Fake Confidence (%)	Prediction
1	4	0.12	8	Real
2	1	0.61	74	Fake
3	0	0.83	91	Fake
4	5	0.09	5	Real
5	2	0.55	68	Fake

(a) Likely Input Features

Two feature families are visible directly in the log. Blink rate is a behavioural/temporal biometric feature: genuine video shows a physiologically normal blink cadence, whereas many face-swap/GAN generation pipelines struggle to reproduce natural, involuntary blinking, so an abnormally low or absent blink rate (frames 2, 3, 5) is a classic deepfake indicator. Texture/artifact score is a spatial, pixel-level forensic feature capturing GAN fingerprint noise, unnaturally smooth or blended skin texture, blending-boundary artifacts around the face swap, and frequency-domain irregularities introduced by upsampling — all of which increase with more heavily manipulated frames. A production system of this kind would typically also draw on supporting features not shown in this log, such as head-pose/landmark jitter across frames, eye-gaze and lighting/shadow consistency, and compression-artifact statistics, but blink rate and texture score are the two the given output explicitly exposes.

(b) Likely Model / Technique

The pattern of two conceptually different, separately-reported features feeding a single fused confidence score suggests a multi-stream deep-learning pipeline rather than one end-to-end network. The texture/artifact score most plausibly comes from a CNN-based forensic classifier (e.g., an XceptionNet- or EfficientNet-style network trained on real vs. manipulated face crops) that specialises in spatial artifacts. The blink rate most plausibly comes from a separate temporal/behavioural module — landmark/eye-aspect-ratio tracking over a short window, possibly summarised by a recurrent network

(RNN/LSTM) — in the spirit of blink-based deepfake-detection methods that exploit the difficulty early generative models had with realistic eye closure. A fusion stage (a small feed-forward network, logistic regression, or a weighted ensemble rule) then combines the two streams into the single Fake Confidence percentage reported per frame.

(c) Deriving the Final Real/Fake Decision from Frame-Level Scores

At the frame level, the prediction is consistent with a simple threshold rule on Fake Confidence — roughly, confidence above 50% is labelled Fake and confidence below 50% is labelled Real, which fits all five rows exactly (8%, 74%, 91%, 5%, 68% → Real, Fake, Fake, Real, Fake). For a whole video, however, a single frame is rarely trusted as the final verdict, since frame 4's momentary “Real” reading sits between two “Fake” frames and is more likely camera/compression noise than genuine evidence the whole clip is authentic. The overall video-level decision is therefore more plausibly derived by aggregating frame-level confidences over time — for example, a moving-average or majority vote over a sliding window of frames, or requiring the average confidence across the full clip (or the fraction of frames flagged Fake) to cross a threshold — before the system commits to a final Real/Fake verdict for the video as a whole.

(d) Improving Reliability with a Swarm/Ensemble Approach

Reliability could be improved by running several independently-specialised detectors in parallel as a “swarm” of expert models — for instance a texture/artifact CNN, a blink/eye-tracking detector, a lip-sync audio-visual consistency detector, and a frequency-domain GAN-fingerprint detector — and combining their individual confidence scores through weighted ensemble voting or stacking rather than relying on a single fused score as at present. This matters because different deepfake-generation methods leave different telltale signatures: a newer generator that has learned to blink naturally might still fail a texture-based test, and vice versa, so a single feature stream can be defeated by improvements in generation technology while an ensemble degrades far more gracefully. As an additional benefit, disagreement among the swarm's members (for example, one detector confident it is real while another is confident it is fake, as borderline frame 5 hints at) can itself be used as an uncertainty signal that automatically routes ambiguous frames to a human reviewer rather than letting the system silently commit to a possibly wrong automated verdict.